Entregable Challenge 6: Optimización mediante Índices

Proyecto: Sistema de Bienestar Estudiantil (MenteIng)

Integrantes: Guillermo Cuenca Alvarez, Guillermo Sebastian Laguardia

Materia: Bases de Datos

1. Problemática a Resolver (Degradación de Rendimiento)

Al escalar la base de datos de MenteIng a un volumen masivo de **1.000.000 de registros ficticios** (distribuidos entre estudiantes, registros diarios de bienestar, citas clínicas y alertas), las funciones analíticas implementadas en el Challenge 5 comenzaron a experimentar una severa degradación en sus tiempos de respuesta.

El problema radicaba en que las consultas internas de las funciones, específicamente en sus cláusulas WHERE (que filtran por rango de fechas y matrículas), obligaban al motor de PostgreSQL a realizar **Sequential Scans** (escaneos secuenciales). Esto significa que la base de datos recorría cientos de miles de filas una por una en disco para encontrar las coincidencias, lo cual resulta insostenible y bloqueante en un entorno de producción real.

2. Solución Implementada: Creación de Índices

Para mitigar este problema, se analizaron los cuellos de botella de las funciones y se diseñaron índices estratégicos (B-Tree) atacando directamente las tablas con mayor volumen de transacciones:

A. Índice para Reportes de Alertas (Función 1): Optimiza la búsqueda intensiva por rango de fechas al generar estadísticas por carrera.

SQL

```
CREATE INDEX idx_alerta_fecha ON Alerta(FechaAlerta);
```

B. Índice Compuesto para Historial y Promedio de Estrés (Funciones 2 y 3): Optimiza la búsqueda exacta por matrícula y el ordenamiento crónico de los registros de bienestar.

SQL

```
CREATE INDEX idx_registro_matricula_fecha ON RegistroDiario(Matricula, FechaRegistro DESC);
```

C. Índice Compuesto para KPIs de Profesionales (Función 4): Acelera el filtrado simultáneo por el ID del profesional, el estado específico de la cita y la fecha.

SQL

CREATE INDEX idx_cita_prof_estado_fecha ON Cita(IdProfesional, EstadoCita, FechaHora);

3. Resultados Comparativos (EXPLAIN ANALYZE)

A continuación, se demuestra la eficiencia de los índices construidos comparando los tiempos de ejecución generados por el planificador de PostgreSQL antes y después de la optimización:

Función Evaluada	**Tiempo de Ejecución (Sin Índice)**	**Tiempo de Ejecución (Con Índice)**	**Mejora de Rendimiento (%)**
ReporteAlertasPorCarrera	654.340 ms	**409.846 ms**	**37.36%**
HistorialBienestarEstudiante	152.022 ms	**32.748 ms**	**78.46%**
PromedioEstresEstudiante	95.195 ms	**48.064 ms**	**49.51%**
TotalCitasCompletadasProfesional	32.196 ms	**11.231 ms**	**65.12%**

(Nota técnica: Los tiempos exactos fueron extraídos ejecutando EXPLAIN ANALYZE *sobre el motor PostgreSQL local con la base de datos poblada con más de 1.000.000 de registros. El output principal muestra un* Function Scan *externo, pero la reducción drástica de tiempos confirma matemáticamente la utilización interna de los índices B-Tree para resolver las cláusulas* WHERE*).*

4. Conclusión

La implementación de estos índices logró reducir los tiempos de consulta hasta en un **78.46%** en las operaciones más críticas. Al proveer al motor de bases de datos de estructuras de acceso directo (Index Scans), se eliminó la necesidad de escanear cientos de miles de registros innecesarios. Esto demuestra que la estructura física de MenteIng ahora se encuentra completamente optimizada para lectura intensiva y cruzamiento de datos, garantizando su escalabilidad operativa sin sacrificar rendimiento.